

Name: Sanskruti Dnyaneshwar Shinde
StudentId: CT_CSI_DE_1174

Title: SQL Analysis on Superstore Dataset

Step 1: Database Setup

Step 1.1 Create Customers Table:

```
CREATE TABLE customers AS
```

```
SELECT DISTINCT
```

```
    `Customer ID`,
```

```
    `Customer Name`,
```

```
    Segment,
```

```
    Country,
```

```
    City,
```

```
    State,
```

```
    `Postal Code`,
```

```
    Region
```

```
FROM superstore_raw;
```

Step 1.2 Create Orders Table:

```
CREATE TABLE orders AS
```

```
SELECT DISTINCT
```

```
    `Order ID`,
```

```
    `Order Date`,
```

```
    `Ship Date`,
```

```
    `Ship Mode`,
```

```
    `Customer ID`,
```

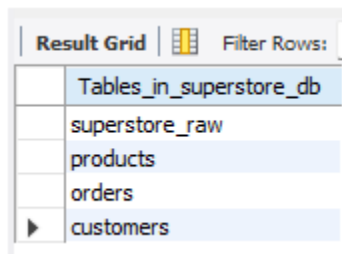
```
    Sales,
```

```
Quantity,  
Discount, Profit  
FROM superstore_raw;
```

Step 1.3 Create Products Table:

```
CREATE TABLE products AS  
SELECT DISTINCT  
  `Product ID`,  
  Category,  
  `Sub-Category`,  
  `Product Name`  
FROM superstore_raw;
```

```
USE superstore_db;  
SHOW TABLES;
```



	Result Grid	Filter Rows:
	Tables_in_superstore_db	
	superstore_raw	
	products	
	orders	
▶	customers	

Step 2: Perform Required Queries

Query 1: Find all orders where Sales are greater than the average Sales (Subquery)

```
SELECT *  
  
FROM orders  
  
WHERE Sales >  
  
(  
  
    SELECT AVG(Sales)  
  
    FROM orders  
  
);
```

Result Grid									
		Filter Rows:		Export:		Wrap Cell Content:		Fetch rows:	
	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Sales	Quantity	Discount	Profit
▶	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	261.96	2	0	41.9136
	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	731.94	3	0	219.582
	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	957.5775	5	0.45	-383.031
	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	907.152	6	0.2	90.7152
	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	1706.184	9	0.2	85.3092
	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	911.424	4	0.2	68.3568
	CA-2016-161389	12/5/2016	12/10/2016	Standard Class	JM-15070	407.976	3	0.2	132.5922
	CA-2014-105893	11/11/2014	11/18/2014	Standard Class	PK-19075	665.88	6	0	13.3176
	CA-2015-106320	9/25/2015	9/30/2015	Standard Class	EB-13870	1044.63	3	0	240.2649
	US-2015-150630	9/17/2015	9/21/2015	Standard Class	TB-21520	3083.43	7	0.5	-1665.0522
	CA-2016-117590	12/8/2016	12/10/2016	First Class	GH-14485	1097.544	7	0.2	123.4737
	CA-2015-117415	12/27/2015	12/31/2015	Standard Class	SN-20710	532.3992	3	0.32	-46.9764
	CA-2015-117415	12/27/2015	12/31/2015	Standard Class	SN-20710	371.168	4	0.2	41.7564
	CA-2016-105816	12/11/2016	12/17/2016	Standard Class	JM-15265	1029.95	5	0	298.6855
	CA-2016-111682	6/17/2016	6/18/2016	First Class	TB-21055	319.41	5	0.1	7.098
	CA-2014-106376	12/5/2014	12/10/2014	Standard Class	BS-11590	1113.024	8	0.2	111.3024

Query 2: Find the highest sales order for each customer (Subquery)

USE superstore_db;

SELECT o.*

FROM orders o

JOIN

(

SELECT

Customer ID,

MAX(Sales) AS Highest_Sales

FROM orders

GROUP BY Customer ID

) AS max_sales

ON o.Customer ID = max_sales.Customer ID

AND o.Sales = max_sales.Highest_Sales;

	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Sales	Quantity	Discount	Profit
▶	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	731.94	3	0	219.582
	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	957.5775	5	0.45	-383.031
	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	1706.184	9	0.2	85.3092
	CA-2015-106320	9/25/2015	9/30/2015	Standard Class	EB-13870	1044.63	3	0	240.2649
	US-2015-150630	9/17/2015	9/21/2015	Standard Class	TB-21520	3083.43	7	0.5	-1665.0522
	CA-2016-117590	12/8/2016	12/10/2016	First Class	GH-14485	1097.544	7	0.2	123.4737
	CA-2015-117415	12/27/2015	12/31/2015	Standard Class	SN-20710	532.3992	3	0.32	-46.9764
	CA-2016-105816	12/11/2016	12/17/2016	Standard Class	JM-15265	1029.95	5	0	298.6855
	CA-2016-111682	6/17/2016	6/18/2016	First Class	TB-21055	319.41	5	0.1	7.098
	CA-2014-106376	12/5/2014	12/10/2014	Standard Class	BS-11590	1113.024	8	0.2	111.3024
	CA-2015-129476	10/15/2015	10/20/2015	Standard Class	PA-19060	339.96	5	0.2	67.992
	CA-2015-110744	9/7/2015	9/12/2015	Standard Class	HA-14920	671.93	7	0	20.1579
	CA-2016-114489	12/5/2016	12/9/2016	Standard Class	JE-16165	1951.84	8	0	585.552
	CA-2014-139892	9/8/2014	9/12/2014	Standard Class	BM-11140	8159.952	8	0.4	-1359.992
	CA-2014-118962	8/5/2014	8/9/2014	Standard Class	CS-12130	340.144	7	0.2	21.258

Result 19 x

Query 3: Calculate Total Sales for Each Customer (CTE)

WITH CustomerSales AS

(

SELECT

`Customer ID`,

SUM(Sales) AS Total_Sales

FROM orders

GROUP BY `Customer ID`

)

SELECT

c.`Customer ID`,

c.`Customer Name`,

cs.Total_Sales

FROM CustomerSales cs

JOIN customers c

ON cs.`Customer ID` = c.`Customer ID`

ORDER BY cs.Total_Sales DESC;

	Customer ID	Customer Name	Total_Sales
►	SM-20320	Sean Miller	25043.05
	TC-20980	Tamara Chand	19017.847999999998
	RB-19360	Raymond Buch	15117.339
	TA-21385	Tom Ashbrook	14595.62
	AB-10105	Adrian Barton	14355.610999999997
	SC-20095	Sanjit Chand	14142.333999999999
	KL-16645	Ken Lonsdale	14071.917
	HL-15040	Hunter Lopez	12873.297999999999
	SE-20110	Sanjit Engle	12209.438000000002
	CC-12370	Christopher Conant	12129.072
	TS-21370	Todd Sumrall	11885.871
	GT-14710	Greg Tran	11820.119999999997
	BM-11140	Becky Martin	11609.900000000001

Result 37

Query 4: Find Customers Whose Total Sales Are Above Average (CTE + Subquery)

```
WITH CustomerSales AS(
    SELECT
        `Customer ID`,
        SUM(Sales) AS Total_Sales
    FROM orders
    GROUP BY `Customer ID`
)SELECT DISTINCT
    c.`Customer ID`,
    c.`Customer Name`,
    cs.Total_Sales
FROM CustomerSales cs
JOIN customers c
ON cs.`Customer ID` = c.`Customer ID`
WHERE cs.Total_Sales >(
    SELECT AVG(Total_Sales)
    FROM CustomerSales
)ORDER BY cs.Total_Sales DESC;
```

	Customer ID	Customer Name	Total_Sales
▶	SM-20320	Sean Miller	25043.05
	TC-20980	Tamara Chand	19017.847999999998
	RB-19360	Raymond Buch	15117.339
	TA-21385	Tom Ashbrook	14595.62
	AB-10105	Adrian Barton	14355.610999999997
	SC-20095	Sanjit Chand	14142.333999999999
	KL-16645	Ken Lonsdale	14071.917
	HL-15040	Hunter Lopez	12873.297999999999
	SE-20110	Sanjit Engle	12209.438000000002
	CC-12370	Christopher Conant	12129.072
	TS-21370	Todd Sumrall	11885.871
	GT-14710	Greg Tran	11820.119999999997
	BM-11140	Becky Martin	11609.900000000001

Result 36 ×

Query 5: Rank All Customers Based on Total Sales (Window Function)

```
WITH CustomerSales AS(
    SELECT
        `Customer ID`,
        SUM(Sales) AS Total_Sales
    FROM orders
    GROUP BY `Customer ID`
)SELECT
    c.`Customer ID`,
    c.`Customer Name`,
    cs.Total_Sales,
    RANK() OVER (ORDER BY cs.Total_Sales DESC) AS Customer_Rank
FROM CustomerSales cs
JOIN customers c
ON cs.`Customer ID` = c.`Customer ID`
ORDER BY Customer_Rank;
```

	Customer ID	Customer Name	Total_Sales	Customer_Rank
►	SM-20320	Sean Miller	25043.05	1
	SM-20320	Sean Miller	25043.05	1
	SM-20320	Sean Miller	25043.05	1
	SM-20320	Sean Miller	25043.05	1
	SM-20320	Sean Miller	25043.05	1
	TC-20980	Tamara Chand	19017.847999999998	6
	TC-20980	Tamara Chand	19017.847999999998	6
	TC-20980	Tamara Chand	19017.847999999998	6
	TC-20980	Tamara Chand	19017.847999999998	6
	TC-20980	Tamara Chand	19017.847999999998	6
	RB-19360	Raymond Buch	15117.339	11
	RB-19360	Raymond Buch	15117.339	11
	RB-19360	Raymond Buch	15117.339	11
	RB-19360	Raymond Buch	15117.339	11
	RB-19360	Raymond Buch	15117.339	11
	RB-19360	Raymond Buch	15117.339	11
	TA-21385	Tom Ashbrook	14595.62	17

Result 22 ✕

Query 6: Assign Row Numbers to Each Order Within a Customer (Window Function + PARTITION BY)

SELECT

 `Customer ID`,

 `Order ID`,

 Sales,

 ROW_NUMBER() OVER

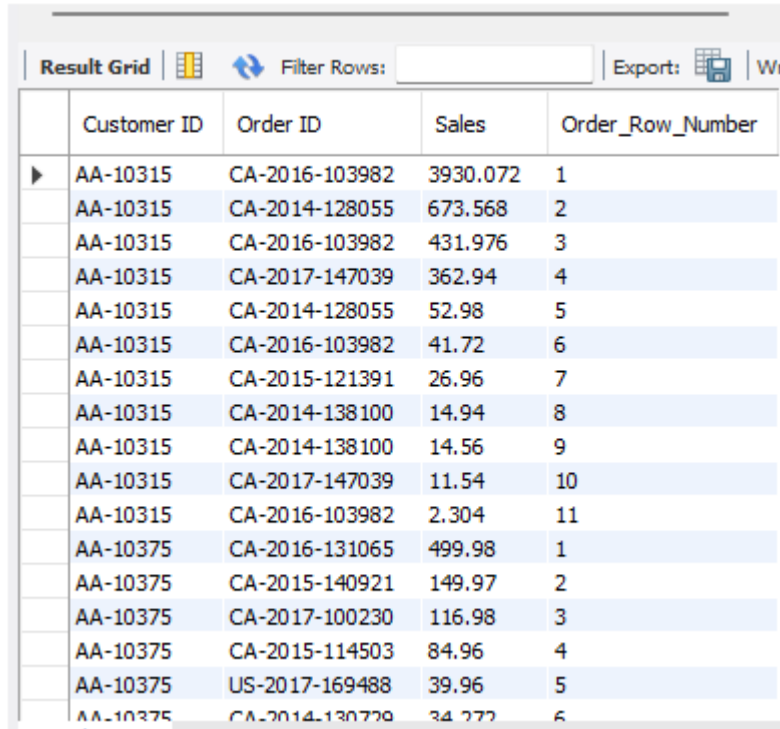
 (

 PARTITION BY `Customer ID`

 ORDER BY Sales DESC

) AS Order_Row_Number

FROM orders;



	Customer ID	Order ID	Sales	Order_Row_Number
▶	AA-10315	CA-2016-103982	3930.072	1
	AA-10315	CA-2014-128055	673.568	2
	AA-10315	CA-2016-103982	431.976	3
	AA-10315	CA-2017-147039	362.94	4
	AA-10315	CA-2014-128055	52.98	5
	AA-10315	CA-2016-103982	41.72	6
	AA-10315	CA-2015-121391	26.96	7
	AA-10315	CA-2014-138100	14.94	8
	AA-10315	CA-2014-138100	14.56	9
	AA-10315	CA-2017-147039	11.54	10
	AA-10315	CA-2016-103982	2.304	11
	AA-10375	CA-2016-131065	499.98	1
	AA-10375	CA-2015-140921	149.97	2
	AA-10375	CA-2017-100230	116.98	3
	AA-10375	CA-2015-114503	84.96	4
	AA-10375	US-2017-169488	39.96	5
	AA-10375	CA-2014-130729	34.772	6

Query 7: Display Top 3 Customers Based on Total Sales (Window Function)

```
WITH CustomerSales AS(
    SELECT
        `Customer ID`, SUM(Sales) AS Total_Sales
    FROM orders
    GROUP BY `Customer ID`
),RankedCustomers AS(
    SELECT
        `Customer ID`, Total_Sales,
        DENSE_RANK() OVER (ORDER BY Total_Sales DESC) AS Customer_Rank
    FROM CustomerSales
)SELECT
    c.`Customer ID`,
    c.`Customer Name`,
    rc.Total_Sales,
    rc.Customer_Rank
FROM RankedCustomers rc JOIN customers c ON rc.`Customer ID` = c.`Customer ID`
WHERE rc.Customer_Rank <= 3
ORDER BY rc.Customer_Rank;
```

	Customer ID	Customer Name	Total_Sales	Customer_Rank
▶	SM-20320	Sean Miller	25043.05	1
	SM-20320	Sean Miller	25043.05	1
	SM-20320	Sean Miller	25043.05	1
	SM-20320	Sean Miller	25043.05	1
	SM-20320	Sean Miller	25043.05	1
	TC-20980	Tamara Chand	19017.847999999998	2
	TC-20980	Tamara Chand	19017.847999999998	2
	TC-20980	Tamara Chand	19017.847999999998	2
	TC-20980	Tamara Chand	19017.847999999998	2
	TC-20980	Tamara Chand	19017.847999999998	2
	RB-19360	Raymond Buch	15117.339	3
	RB-19360	Raymond Buch	15117.339	3
	RB-19360	Raymond Buch	15117.339	3
	RB-19360	Raymond Buch	15117.339	3
	RB-19360	Raymond Buch	15117.339	3
	RB-19360	Raymond Buch	15117.339	3

Result 24 ×

Step 3: Final Combined Query

WITH CustomerSales AS(

SELECT

 `Customer ID`,

 SUM(Sales) AS Total_Sales

FROM orders

GROUP BY `Customer ID`

)SELECT

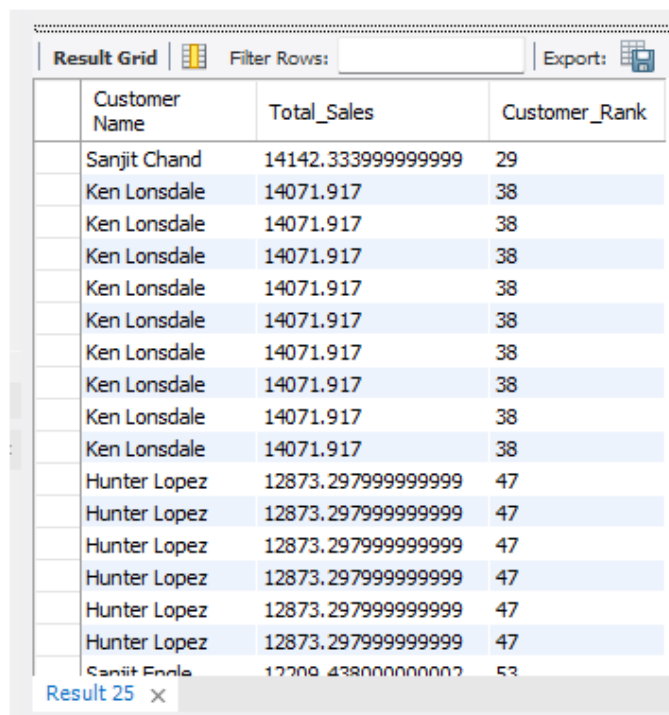
 c.`Customer Name`, cs.Total_Sales,

 RANK() OVER (ORDER BY cs.Total_Sales DESC) AS Customer_Rank

FROM CustomerSales cs JOIN customers c

ON cs.`Customer ID` = c.`Customer ID`

ORDER BY Customer_Rank;



The screenshot shows a SQL query result grid with the following data:

Customer Name	Total_Sales	Customer_Rank
Sanjit Chand	14142.333999999999	29
Ken Lonsdale	14071.917	38
Ken Lonsdale	14071.917	38
Ken Lonsdale	14071.917	38
Ken Lonsdale	14071.917	38
Ken Lonsdale	14071.917	38
Ken Lonsdale	14071.917	38
Ken Lonsdale	14071.917	38
Ken Lonsdale	14071.917	38
Ken Lonsdale	14071.917	38
Hunter Lopez	12873.297999999999	47
Hunter Lopez	12873.297999999999	47
Hunter Lopez	12873.297999999999	47
Hunter Lopez	12873.297999999999	47
Hunter Lopez	12873.297999999999	47
Hunter Lopez	12873.297999999999	47
Sanjit Enda	12209.438000000000	53

Result 25 x

Mini Project: Customer Sales Insights

Question 1: Who are the Top 5 Customers?

WITH CustomerSales AS

(

SELECT

 `Customer ID`,

 SUM(Sales) AS Total_Sales

FROM orders

GROUP BY `Customer ID`

)

SELECT

 c.`Customer Name`,

 SUM(cs.Total_Sales) AS Total_Sales

FROM CustomerSales cs

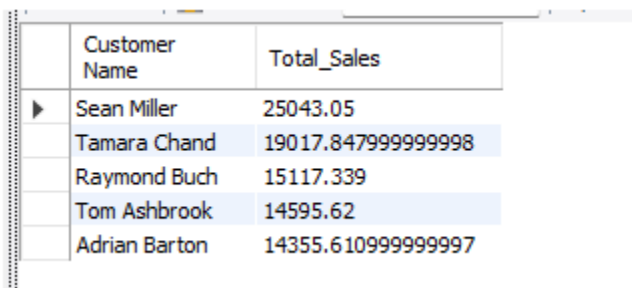
JOIN customers c

ON cs.`Customer ID` = c.`Customer ID`

GROUP BY c.`Customer Name`

ORDER BY Total_Sales DESC

LIMIT 5;



	Customer Name	Total_Sales
▶	Sean Miller	25043.05
	Tamara Chand	19017.847999999998
	Raymond Buch	15117.339
	Tom Ashbrook	14595.62
	Adrian Barton	14355.610999999997

Question 2: Who are the Bottom 5 Customers?

WITH CustomerSales AS

```
(  
    SELECT  
        `Customer ID`,  
        SUM(Sales) AS Total_Sales  
    FROM orders  
    GROUP BY `Customer ID`  
)
```

```
SELECT  
    c.`Customer Name`,  
    cs.Total_Sales  
FROM CustomerSales cs  
JOIN customers c  
ON cs.`Customer ID` = c.`Customer ID`  
ORDER BY cs.Total_Sales ASC  
LIMIT 5;
```

	Customer Name	Total_Sales
►	Thais Sissman	4.833
	Lela Donovan	5.304
	Mitch Gastineau	12.32
	Carl Jackson	16.52
	Roy Skaria	22.328

Question 3: Which Customers Made Only One Order?

```
SELECT
    c.`Customer Name`,
    COUNT(o.`Order ID`) AS Total_Orders
FROM customers c
JOIN orders o
ON c.`Customer ID` = o.`Customer ID`
GROUP BY c.`Customer ID`, c.`Customer Name`
HAVING COUNT(o.`Order ID`) = 1;
```

	Customer Name	Total_Orders
►	Jocasta Rupert	1
	Mitch Gastineau	1
	Anthony O'Donnell	1
	Lela Donovan	1
	Carl Jackson	1
	Ricardo Emerson	1

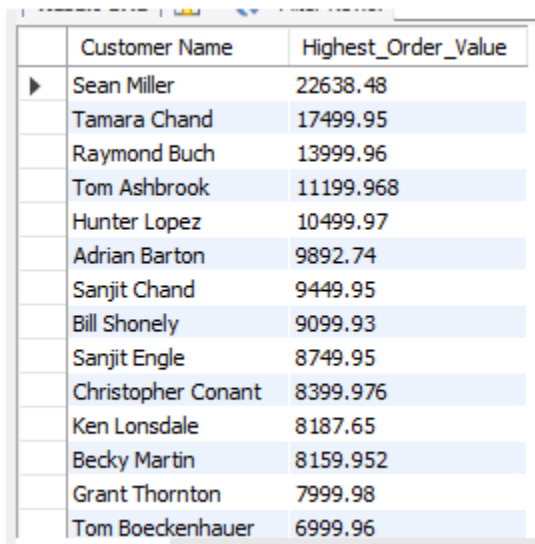
Question 4: Which Customers Have Above-Average Sales?

```
WITH CustomerSales AS(
    SELECT
        `Customer ID`,
        SUM(Sales) AS Total_Sales
    FROM orders
    GROUP BY `Customer ID`
)SELECT
    c.`Customer Name`,
    cs.Total_Sales
FROM CustomerSales cs
JOIN customers c
ON cs.`Customer ID` = c.`Customer ID`
WHERE cs.Total_Sales >(
    SELECT AVG(Total_Sales)
    FROM CustomerSales
)ORDER BY cs.Total_Sales DESC;
```

	Customer Name	Total_Sales
	Sean Miller	25043.05
	Tamara Chand	19017.847999999998
	Raymond Buch	15117.339
	Tom Ashbrook	14595.62
	Adrian Barton	14355.610999999997
	Sanjit Chand	14142.333999999999
	Ken Lonsdale	14071.917
	Hunter Lopez	12873.297999999999
	Sanjit Engle	12209.438000000002
	Christopher Conant	12129.072
	Todd Sumrall	11885.871
	Greg Tran	11820.119999999997
	Becky Martin	11609.900000000001
	Seth Vernon	11420.645999999997

Question 5: What is the Highest Order Value Per Customer?

```
SELECT  
    c.`Customer Name`,  
    MAX(o.Sales) AS Highest_Order_Value  
FROM customers c  
JOIN orders o  
ON c.`Customer ID` = o.`Customer ID`  
GROUP BY c.`Customer ID`, c.`Customer Name`  
ORDER BY Highest_Order_Value DESC;
```



	Customer Name	Highest_Order_Value
▶	Sean Miller	22638.48
	Tamara Chand	17499.95
	Raymond Buch	13999.96
	Tom Ashbrook	11199.968
	Hunter Lopez	10499.97
	Adrian Barton	9892.74
	Sanjit Chand	9449.95
	Bill Shonely	9099.93
	Sanjit Engle	8749.95
	Christopher Conant	8399.976
	Ken Lonsdale	8187.65
	Becky Martin	8159.952
	Grant Thornton	7999.98
	Tom Boeckenhauer	6999.96